COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/02/26	**DATE SUBMITTED:**	10/02/26
NAME:	Aridiodon, Gabriel Dwyane D.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO **HOA 10.1 Graphs**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

```
#include <iostream>
// stores adjacency list items
struct adjNode
{
    int val, cost;
    adjNode *next;
};
// structure to store edges
struct graphEdge
{
    int start_ver, end_ver, weight;
};
class DiaGraph
{
    // insert new nodes into adjacency list from given graph
    adjNode *getAdjListNode(int value, int weight, adjNode *head)
    {
        adjNode *newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head

        return newNode;
    }
    int N; // number of nodes in the graph
public:
    adjNode **head; // adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N)
    {
        // allocate new node
```

```
        head = new adjNode *[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++)
        {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode *newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }
    // Destructor
    ~DiaGraph()
    {
        for (int i = 0; i < N; i++)
            delete[] head[i];
        delete[] head;
    }
};
// print all adjacent vertices of given vertex
void display_AdjList(adjNode *ptr, int i)
{
    while (ptr != nullptr)
    {
        std::cout << "(" << i << ", " << ptr->val
                  << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0, 1, 2},
        {0, 2, 4},
```

```
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}};

    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout << "Graph adjacency list " << "(start_vertex, end_vertex, weight):"
<< std::endl;
    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

Output:

```
Graph adjacency list (start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Observation:

The adjacency list stores only the edges that actually exist,Each new node is inserted at the head of its list, so the edges of vertex 0 print in reverse order of insertion: (0, 2, 4) comes before (0, 1, 2). Vertex 5 has no outgoing edges, so its list is empty.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

```
#include <string>
#include <vector>
#include <iostream>
```

```cpp
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
```

```
    Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

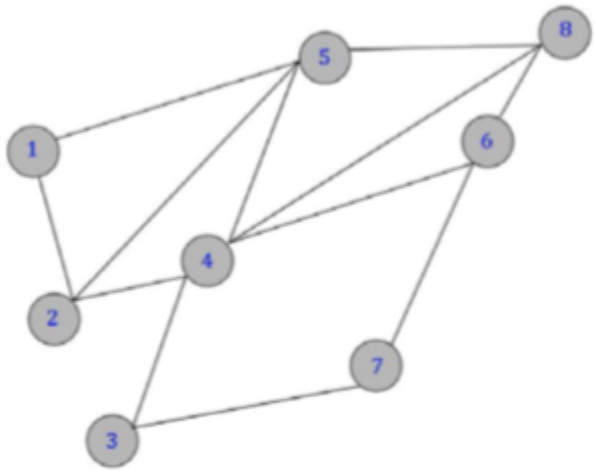
```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
```

```
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}
int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Graph for testing:



Output:

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

Observation:
DFS uses a stack so it follows one path as deep as it can (1 -> 5 -> 8 -> 6 -> 7) before it backtracks then visits 3->4->2.

B.2. Breadth-First Search

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>
template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
```

```
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}
    auto vertices() const
    {
        return V;
    }
    auto &edges() const
    {
        return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
```

```
    }
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

template <typename Y>
friend std::ostream &operator<<(std::ostream &os, const Graph<Y> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;

};

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
```

```
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

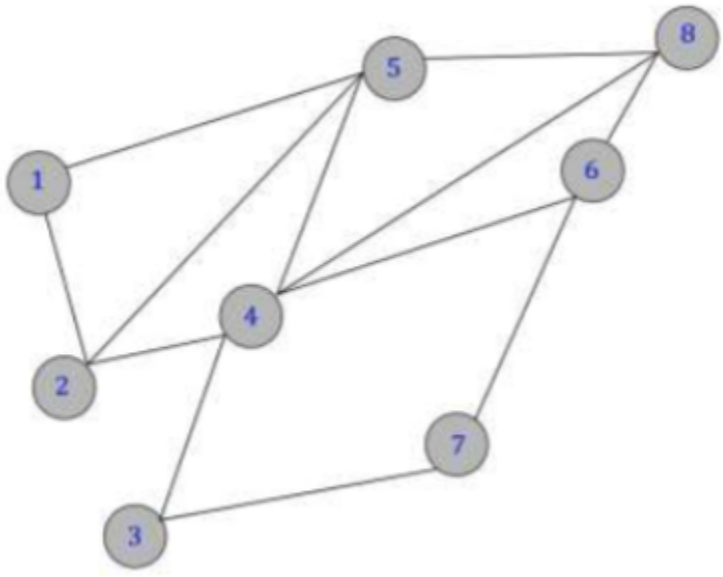
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<T>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}
int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

Output:

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Graph for testing:



Observation:
BFS uses a queue, so it visits vertices level by level. Vertex 1 comes first, then its neighbors 2 and 5, then their unvisited neighbors 4 and 8, and finally 3, 6, and 7.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

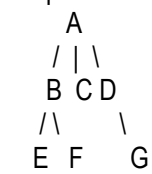
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

Depth-First Search (DFS). The person follows one path as far as it goes, backtracks to the most recent vertex that still has unvisited neighbors, and continues from there. This is exactly DFS's "go deep, then backtrack" behavior. It is implemented with a stack or recursion, and the call stack or explicit stack remembers where to backtrack to. BFS would not work here because it visits all neighbors of a vertex first, so it never commits to one path.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

Graphical comparison, using this tree:



DFS
Tree Equivalent: Preorder
Visit order: A, B, E, F, C, D, G

BFS
Tree Equivalent: Level-order
Visit order: A, B, C, D, E, F, G

DFS Pseudo

```
DFS(vertex):
    mark vertex as visited
    visit vertex

    for each neighbor of vertex:
        if neighbor is not visited:
            DFS(neighbor)
```

BFS Pseudo

```
BFS(start):
    create a queue
    mark start as visited
    add start to queue

    while queue is not empty:
        remove vertex from queue
        visit vertex

        for each neighbor of vertex:
            if neighbor is not visited:
                mark neighbor as visited
                add neighbor to queue
```

```cpp
#include <iostream>
#include <queue>
using namespace std;

int graph[7][7] = {
    // A  B  C  D  E  F  G
    { 0, 1, 1, 1, 0, 0, 0 }, // A
    { 0, 0, 0, 0, 1, 1, 0 }, // B
    { 0, 0, 0, 0, 0, 0, 0 }, // C
    { 0, 0, 0, 0, 0, 0, 1 }, // D
    { 0, 0, 0, 0, 0, 0, 0 }, // E
    { 0, 0, 0, 0, 0, 0, 0 }, // F
    { 0, 0, 0, 0, 0, 0, 0 }  // G
};

bool visited[7];

// DFS
void DFS(int vertex)
{
    visited[vertex] = true;
    cout << char('A' + vertex) << " ";
```

```
    for (int i = 0; i < 7; i++)
    {
        if (graph[vertex][i] == 1 && !visited[i])
        {
            DFS(i);
        }
    }
}

// BFS
void BFS(int start)
{
    bool visited[7] = {false};
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty())
    {
        int vertex = q.front();
        q.pop();

        cout << char('A' + vertex) << " ";

        for (int i = 0; i < 7; i++)
        {
            if (graph[vertex][i] == 1 && !visited[i])
            {
                visited[i] = true;
                q.push(i);
            }
        }
    }
}

int main()
```

```
{
    cout << "DFS: ";
    DFS(0);

    cout << "\nBFS: ";
    BFS(0);

    return 0;
}
```

Output:

```
DFS: A B E F C D G
BFS: A B C D E F G
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

BFS uses a queue because it follows the First-In, First-Out (FIFO) principle. The vertex that is discovered first is processed first.

4. How many times can a node be visited in the BFS?

Exactly once. The visited set is checked before a vertex is processed, so each vertex is added to the visit order a single time. When a duplicate is dequeued, the visited check skips it, so it is never visited again.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.